

5 Mapping persistent classes

This chapter covers

- Understanding entities and value types
- Mapping entity classes with identity
- Controlling entity-level mapping options

This chapter presents some fundamental mapping options and explains how to map entity classes to SQL tables. This is essential knowledge for structuring classes in an application, no matter whether you work with Hibernate, Spring Data JPA, or some other persistence framework that implement the JPA specification. We'll demonstrate and analyze how you can handle database identity and primary keys, and how you can use various other metadata settings to customize how Hibernate or Spring Data JPA using Hibernate as a persistence provider will load and store instances of your domain model classes.

Spring Data JPA, as a data access abstraction, sits on top of a JPA provider (such as Hibernate) and will significantly reduce the boilerplate code required to interact with the database. That is why, once the mapping of persistent classes is made, it can be used both from Hibernate and from Spring Data JPA. Our examples will demonstrate this, and all our mapping examples will use JPA annotations.

Before we look at mapping, though, we'll define the essential distinction between entities and value types and explain how you should approach the object/relational mapping of your domain model. The role of the engineer is to make the connection between the *application domain*, which is the environment of the problem that the system needs to address, and the *solution domain*, which is the software and the technologies that will build this system. In figure 5.1, the application domain is represented by the application domain model (the real entities), while the solution domain is represented by the system model (the objects from the software application).

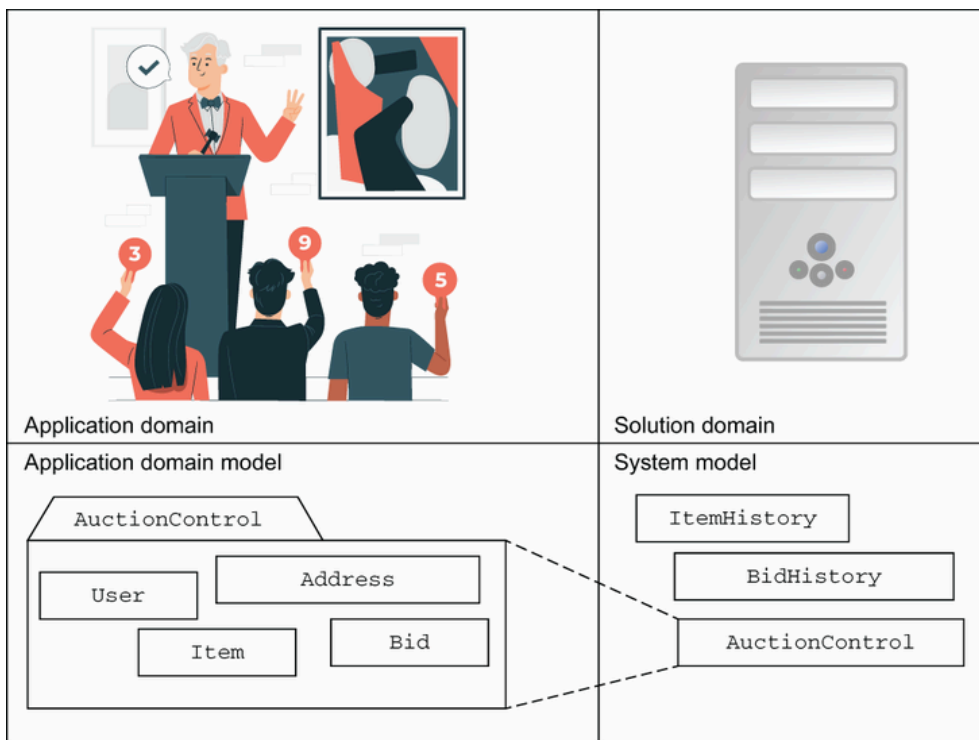


Figure 5.1 The different domains and models to be connected

5.1 Understanding entities and value types

When you look at your domain model, you'll notice a difference between classes: some of the types seem more important, representing first-class business objects (the term *object* is used here in its natural sense). Some examples are the `Item`, `Category`, and `User` classes: these are entities in the real world that you're trying to represent (refer to figure 3.3 for a view of the example domain model). Other types present in your domain model, such as `Address`, seem less important. In this section, we'll look at what it means to use fine-grained domain models and make the distinction between entities and value types.

5.1.1 Fine-grained domain models

A major objective of Hibernate and Spring Data JPA using Hibernate as a persistence provider is providing support for fine-grained and rich domain models. It's one reason we work with POJOs (Plain Old Java Objects)—regular Java objects not bound to any framework. In crude terms, *fine-grained* means having more classes than tables.

For example, a user may have a home address in your domain model. In the database, you may have a single `USERS` table with the columns `HOME_STREET`, `HOME_CITY`, and `HOME_ZIPCODE`. (Remember the problem of SQL types we discussed in section 1.2.1?) In the domain model, you could use the same approach, representing the address as three string-valued properties of the `User` class. But it's much better to model this using an `Address` class, where `User` has a `homeAddress` property. This

domain model achieves improved cohesion and greater code reuse, and it's more understandable than SQL with inflexible type systems.

JPA emphasizes the usefulness of fine-grained classes for implementing type safety and behavior. For example, many people model an email address as a string-valued property of `User`. However, a more sophisticated approach is to define an `EmailAddress` class, which adds higher-level semantics and behavior. It may provide a `prepareMail()` method (but it shouldn't have a `sendMail()` method, because you don't want your domain model classes to depend on the mail subsystem).

This granularity problem leads us to a distinction of central importance in ORM. In Java, all classes are of equal standing—all instances have their own identity and lifecycle. When you introduce persistence, some instances may not have their own identity and lifecycle but instead depend on others. Let's walk through an example.

5.1.2 Defining application concepts

Suppose two people live in the same house, and they both register user accounts in `CaveatEmptor`. Let's call them John and Jane. An instance of `User` represents each account. Because you want to load, save, and delete these `User` instances independently, `User` is an entity class and not a value type. Finding entity classes is easy.

The `User` class has a `homeAddress` property; it's an association with the `Address` class. Do both `User` instances have a runtime reference to the same `Address` instance, or does each `User` instance have a reference to its own `Address`? Does it matter that John and Jane live in the same house?

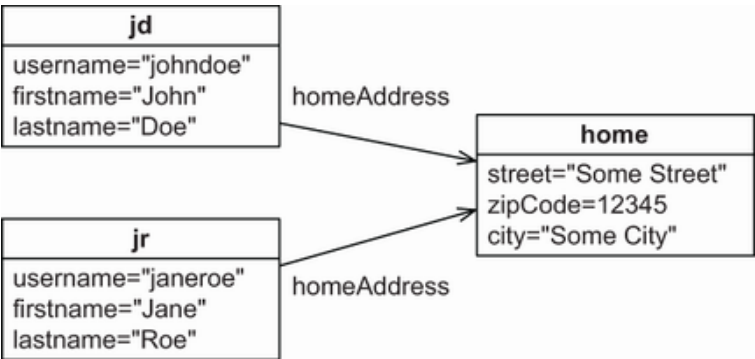


Figure 5.2 Two `User` instances have a reference to a single `Address`.

In figure 5.2, you can see how two `User` instances share a single `Address` instance representing their home address (this is a UML object diagram, not a class diagram). If `Address` is supposed to support shared runtime references, it's an entity type. The `Address` instance has its own

life. You can't delete it when John removes his `User` account—Jane might still have a reference to the `Address`.

Now let's look at the alternative model, where each `User` has a reference to its own `homeAddress` instance, as shown in figure 5.3. In this case, you can make an instance of `Address` dependent on an instance of `User`: you make it a value type. When John removes his `User` account, you can safely delete his `Address` instance. Nobody else will hold a reference to it.

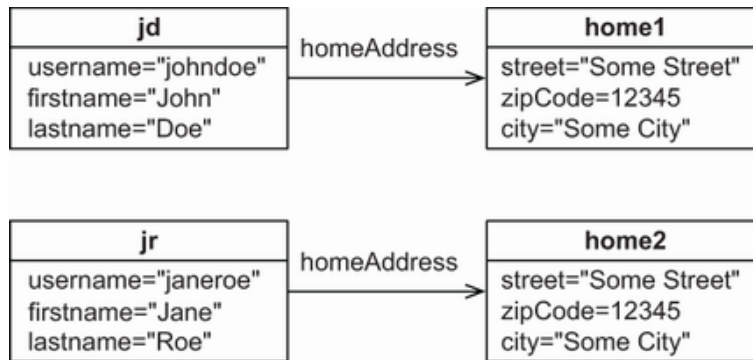


Figure 5.3 Two `User` instances each have their own dependent `Address`.

Hence, we can make the following essential distinction:

- *Entity type*—You can retrieve an instance of an *entity type* using its persistent identity; for example, a `User`, `Item`, or `Category` instance. A reference to an entity instance (a pointer in the JVM) is persisted as a reference in the database (a foreign key–constrained value). An entity instance has its own lifecycle; it can exist independently of any other entity. You map selected classes of your domain model as entity types.
- *Value type*—An instance of a *value type* has no persistent identifier property; it belongs to an entity instance, and its lifespan is bound to the owning entity instance. A value-type instance doesn't support shared references. You can map your own domain model classes as value types; for example, `Address` and `MonetaryAmount`.

If you read the JPA specification, you'll find the same concepts, but value types are called *basic property types* or *embeddable classes* in JPA. We'll come back to this in the next chapter.

Identifying entities and value types in your domain model isn't an ad hoc task but follows a certain procedure.

5.1.3 Distinguishing entities and value types

You may find it helpful to add stereotype information to your UML class diagrams so you can immediately recognize entities and value types (stereotypes are a UML extensibility mechanism). This practice will also force you to think about this distinction for all your classes, which is the first step toward an optimal mapping and well-performing persistence layer. Figure 5.4 shows an example, with the stereotype information in the double angle brackets.

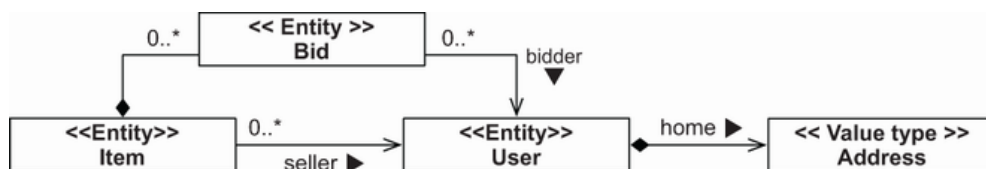


Figure 5.4 Diagramming stereotypes for entities and value types

The `Item` and `User` classes are obvious entities. They each have their own identity, their instances have references from many other instances (shared references), and they have independent lifespans.

Marking `Address` as a value type is also easy: a single `User` instance references a particular `Address` instance. You know this because the association has been created as a composition, with the `User` instance being fully responsible for the lifecycle of the referenced `Address` instance. Therefore, `Address` instances can't be referenced by anyone else and don't need their own identity.

The `Bid` class could be a problem. In object-oriented modeling, this is marked as a composition (the association between `Item` and `Bid` with the full diamond). Composition is a type of association where an object can only exist as part of a container. If the container is destroyed, then the included object is also destroyed. Thus, an `Item` is the owner of its `Bid` instances and holds a collection of references. The `Bid` instances cannot exist without the `Item`. At first, this seems reasonable, because bids in an auction system are useless when the item they were made for disappears.

But what if a future extension of the domain model requires a `User#bids` collection containing all bids made by a particular `User`? Right now, the association between `Bid` and `User` is unidirectional; a `Bid` has a `bidder` reference. What if this was bidirectional?

In that case, you would have to deal with possible shared references to `Bid` instances, so the `Bid` class would need to be an entity. It has a de-

pendent lifecycle, but it must have its own identity to support (future) shared references.

You'll often find this kind of mixed behavior, but your first reaction should be to make everything a value-typed class and promote it to an entity only when absolutely necessary. `Bid` is a value type because its identity is defined by `Item` and `User`. This does not necessarily mean that it won't live in its own table. Try to simplify your associations; persistent collections, for example, frequently add complexity without offering any advantages. Instead of mapping the `Item#bids` and `User#bids` collections, you can write queries to obtain all the bids for an `Item` and those made by a particular `User`. The associations in the UML diagram will point from the `Bid` to the `Item` and `User`, unidirectionally, and not the other way. The stereotype on the `Bid` class would then be `<<Value type>>`. We'll come back to this point in chapter 8.

Next, you can take your domain model diagram and implement POJOs for all entities and value types. You'll have to take care of three things:

- *Shared references*—Avoid shared references to value type instances when you write your POJO classes. For example, make sure only one `User` can reference an `Address`. You can make `Address` immutable with no public `setUser()` method and enforce the relationship with a public constructor that has a `User` argument. Of course, you'll still need a no-argument, probably protected constructor, as we discussed in chapter 3, so Hibernate or Spring Data JPA can also create an instance.
- *Lifecycle dependencies*—If a `User` is deleted, its `Address` dependency will have to be deleted as well. Persistence metadata will include cascading rules for all such dependencies, so Hibernate, Spring Data JPA, or the database can take care of removing the obsolete `Address`. You must design your application procedures and user interface to respect and expect such dependencies—write your domain model POJOs accordingly.
- *Identity*—Entity classes need an identifier property in almost all cases. Value type classes (and, of course, JDK classes such as `String` and `Integer`) don't have an identifier property because instances are identified through the owning entity.

We'll come back to references, associations, and lifecycle rules when we discuss more advanced mappings in later chapters. Object identity and identifier properties are our next topic.

5.2 Mapping entities with identity

Mapping entities with identity requires you to understand Java identity and equality. Once you know that, we can walk through an entity class example and its mapping, discuss terms like *database identity*, and look at how JPA manages identity. After that, we'll be able to dig in deeper and select a primary key, configure key generators, and finally go through identifier generator strategies.

5.2.1 Understanding Java identity and equality

Java developers understand the difference between Java object identity and equality. Object identity (`==`) is a notion defined by the Java virtual machine. Two references are identical if they point to the same memory location.

Object equality, on the other hand, is a notion defined by a class's `equals()` method, sometimes also referred to as *equivalence*.

Equivalence means two different (non-identical) instances have the same value—the same state. If you have a stack of freshly printed books of the same kind, and you have to choose one of them, it means you will have to choose one from several non-identical but equivalent objects.

Two different instances of `String` are equal if they represent the same sequence of characters, even though each has its own location in the memory space of the virtual machine. (If you're a Java guru, we acknowledge that `String` is a special case. Assume we used a different class to make the same point.)

Persistence complicates this picture. With object/relational persistence, a persistent instance is an in-memory representation of a particular row (or rows) of a database table (or tables). Along with Java identity and equality, we define *database identity*. You now have three methods for distinguishing references:

- *Object identity*—Objects are identical if they occupy the same memory location in the JVM. This can be checked with the `a == b` operator. This concept is known as *object identity*.
- *Object equality*—Objects are equal if they have the same state, as defined by the `a.equals(Object b)` method. Classes that don't explicitly override this method inherit the implementation defined by `java.lang.Object`, which compares object identity with `==`. This concept is known as *object equality*. As you'll probably recall, the properties of object equality are reflexivity, symmetry, and transitivity. One thing they imply is that if `a == b`, then both `a.equals(b)` and `b.equals(a)` should be true.

- *Database identity*—Objects stored in a relational database are identical if they share the same table and primary key value. This concept, mapped into the Java space, is known as *database identity*.

We now need to look at how database identity relates to object identity and how we can express database identity in the mapping metadata. As an example, you'll map an entity of a domain model.

5.2.2 A first entity class and mapping

The `@Entity` annotation isn't enough to map a persistent class. You'll also need an `@Id` annotation, as shown in the following listing (see the `generator` folder for the source code).

NOTE To execute the examples from the source code, you'll first need to run the `Ch05.sql` script.

Listing 5.1 Mapped `Item` entity class with an identifier property

```
Path: Ch05/generator/src/main/java/com/manning/javapersistence/ch05/mc
➡ /Item.java
```

```
@Entity
\1 Item {
    @Id
    @GeneratedValue(generator = "ID_GENERATOR")
    private Long id;
    public Long getId() {
        return id;
    }
}
```

This is the most basic entity class, marked as “persistence capable” with the `@Entity` annotation and with an `@Id` mapping for the database identifier property. The class maps by default to a table named `ITEM` in the database schema.

Every entity class has to have an `@Id` property; it's how JPA exposes database identity to the application. We haven't shown the identifier property in our diagrams, but we assume that each entity class has one. In our examples, we always name the identifier property `id`. This is a good practice for your own projects; use the same identifier property name for all your domain model entity classes. If you specify nothing else, this property maps to a primary key column named `ID` in the table in your database schema.

Hibernate and Spring Data JPA will use the field to access the identifier property value when loading and storing items, not getter or setter methods. Because `@Id` is on a field, Hibernate or Spring Data JPA will enable every field of the class as a persistent property by default. The rule in JPA is this: if `@Id` is on a field, the JPA provider will access fields of the class directly and consider all fields to be part of the persistent state by default. In our experience, field access is often a better choice than using an accessor, because it gives you more freedom for accessor method design.

Should you have a public getter method for the identifier property?

Applications often use database identifiers as convenient handles for particular instances, even outside the persistence layer. For example, it's common for web applications to display the results of a search to the user as a list of summaries. When the user selects a particular element, the application may need to retrieve the selected item, and it's common to use a lookup by identifier for this purpose—you've probably already used identifiers this way, even in applications that rely on JDBC.

Should you have a setter method? Primary key values never change, so you shouldn't allow the identifier property value to be modified.

Hibernate and Spring Data JPA using Hibernate as a provider won't update a primary key column, and you shouldn't expose a public identifier setter method on an entity.

The Java type of the identifier property, `java.lang.Long` in the previous example, depends on the primary key column type in the `ITEM` table and how key values are produced. This brings us to the `@GeneratedValue` annotation, and to primary keys in general.

5.2.3 Selecting a primary key

The database identifier of an entity is mapped to a table's primary key, so let's first get some background on primary keys without worrying about mappings. Take a step back and think about how you identify entities.

A *candidate key* is a column or set of columns that you could use to identify a particular row in a table. To become the primary key, a candidate key must satisfy the following requirements:

- The value of any candidate key column is never null. You can't identify something with data that is unknown, and there are no nulls in the relational model. Some SQL products allow you to define (composite) primary keys with nullable columns, so you must be careful.
- The value of the candidate key column (or columns) is a unique value for any row.

- The value of the candidate key column (or columns) never changes; it's immutable.

Must primary keys be immutable?

The relational model requires that a candidate key must be unique and irreducible (no subset of the key attributes has the uniqueness property). Beyond that, picking a candidate key as *the* primary key is a matter of taste. But Hibernate and Spring Data JPA expect a candidate key to be immutable when it's used as the primary key. Hibernate and Spring Data JPA with Hibernate as a provider don't support updating primary key values with an API; if you try to work around this requirement, you'll run into problems with Hibernate's caching and dirty-checking engine. If your database schema relies on updatable primary keys (and maybe uses `ON UPDATE CASCADE` foreign key constraints), you must change the schema before it will work with Hibernate or Spring Data JPA using Hibernate as provider.

If a table has only one identifying attribute, it becomes, by definition, the primary key. But several columns or combinations of columns may satisfy these properties for a particular table; you can choose between candidate keys to decide on the best primary key for the table. You should declare candidate keys that are not chosen as the primary key as unique keys in the database if their values are indeed unique (but maybe not immutable).

Many legacy SQL data models use natural primary keys. A *natural key* is a key with business meaning: an attribute or combination of attributes that is unique by virtue of its business semantics. Examples of natural keys are the US Social Security Number and Australian Tax File Number. Distinguishing natural keys is simple: if a candidate key attribute has meaning outside the database context, it's a natural key, regardless of whether it's automatically generated. Think about the application's users: if they refer to a key attribute when talking about and working with the application, it's a natural key: "Can you send me the pictures of item #A23-abc?"

Experience has shown that natural primary keys usually cause problems in the end. A good primary key must be unique, immutable, and never null. Few entity attributes satisfy these requirements, and some that do can't be efficiently indexed by SQL databases (although this is an implementation detail and shouldn't be the deciding factor for or against a particular key). You should also make certain that a candidate key definition never changes throughout the lifetime of the database. Changing the value (or even the definition) of a primary key and all foreign keys that refer

to it is a frustrating task. Expect your database schema to survive decades, even if your application won't.

Furthermore, you can often only find natural candidate keys by combining several columns in a *composite* natural key. These composite keys, although certainly appropriate for some schema artifacts (like a link table in a many-to-many relationship), potentially make maintenance, ad hoc queries, and schema evolution much more difficult.

For these reasons, we strongly recommend that you add synthetic identifiers, also called *surrogate keys*. Surrogate keys have no business meaning—they have unique values generated by the database or application. Application users ideally won't see or refer to these key values; they're part of the system's internals. Introducing a surrogate key column is also appropriate in the common situation where there are no candidate keys. In other words, almost every table in your schema should have a dedicated surrogate primary key column with only this purpose.

There are several well-known approaches to generating surrogate key values. The aforementioned `@GeneratedValue` annotation is how you configure this.

5.2.4 Configuring key generators

The `@Id` annotation is required to mark the identifier property of an entity class. Without the `@GeneratedValue` next to it, the JPA provider assumes that you'll take care of creating and assigning an identifier value before you save an instance. We call this an *application-assigned* identifier. Assigning an entity identifier manually is necessary when you're dealing with a legacy database or natural primary keys.

Usually you'll want the system to generate a primary key value when you save an entity instance, so you can write the `@GeneratedValue` annotation next to `@Id`. JPA standardizes several value-generation strategies with the `javax.persistence.GenerationType` enum, which you select with `@GeneratedValue(strategy = ...)`:

- `GenerationType.AUTO`—Hibernate (or Spring Data JPA using Hibernate as a persistence provider) picks an appropriate strategy, asking the SQL dialect of your configured database what is best. This is equivalent to `@GeneratedValue()` without any settings.
- `GenerationType.SEQUENCE`—Hibernate (or Spring Data JPA using Hibernate as a persistence provider) expects (and creates, if you use the tools) a sequence named `HIBERNATE_SEQUENCE` in your database. The sequence will be called separately before every `INSERT`, producing sequential numeric values.

- `GenerationType.IDENTITY` —Hibernate (or Spring Data JPA using Hibernate as a persistence provider) expects (and creates in table DDL) a special auto-incremented primary key column that automatically generates a numeric value on `INSERT` in the database.
- `GenerationType.TABLE` —Hibernate (or Spring Data JPA using Hibernate as a persistence provider) will use an extra table in your database schema that holds the next numeric primary key value, with one row for each entity class. This table will be read and updated before `INSERT`s. The default table name is `HIBERNATE_SEQUENCES` with the columns `SEQUENCE_NAME` and `NEXT_VALUE`.

Although `AUTO` seems convenient, you'll sometimes need more control over how IDs are created, so you usually should instead explicitly configure a primary key generation strategy. Most applications work with database sequences, but you may want to customize the name and other settings of the database sequence. Therefore, instead of picking one of the JPA strategies, you can map the identifier with `@GeneratedValue` (`generator = "ID_GENERATOR"`), as shown in listing 5.1. This is a *named* identifier generator; you are now free to set up the `ID_GENERATOR` configuration independently of your entity classes.

JPA has two built-in annotations you can use to configure named generators: `@javax.persistence.SequenceGenerator` and `@javax.persistence.TableGenerator`. With these annotations, you can create a named generator with your own sequence and table names. As usual with JPA annotations, you can unfortunately only use them at the top of a (maybe otherwise empty) class and not in a `package-info.java` file.

For this reason, and because the JPA annotations don't give you access to the full Hibernate feature set, we prefer the native `@org.hibernate.annotations.GenericGenerator` annotation as an alternative. It supports all Hibernate identifier generator strategies and their configuration details. Unlike the rather limited JPA annotations, you can use the Hibernate annotation in a `package-info.java` file, typically in the same package as your domain model classes. The following listing shows a recommended configuration, which can also be found in the `generator` folder.

Listing 5.2 Hibernate identifier generator configured as package-level metadata

```
Path: Ch05/generator/src/main/java/com/manning/javapersistence/ch05
➡ /package-info.java

@org.hibernate.annotations.GenericGenerator(
    name = "ID_GENERATOR",
```

```

strategy = "enhanced-sequence",                                ❸
parameters = {
    @org.hibernate.annotations.Parameter(
        name = "sequence_name",                                ❹
        value = "JPWHSD_SEQUENCE"
    ),
    @org.hibernate.annotations.Parameter(
        name = "initial_value",                                ❺
        value = "1000"
    )
}
})

```

❸ The `enhanced-sequence` strategy produces sequential numeric values. If your SQL dialect supports sequences, Hibernate (or Spring Data JPA using Hibernate as a persistence provider) will use an actual database sequence. If your DBMS doesn't support native sequences, Hibernate (or Spring Data JPA using Hibernate as a persistence provider) will manage and use an extra “sequence table,” simulating the behavior of a sequence. This gives you real portability: the generator can always be called before performing an SQL `INSERT`, unlike, for example, auto-increment identity columns, which produce a value on `INSERT` that has to be returned to the application afterward.

❹ You can configure the `sequence_name`. Hibernate (or Spring Data JPA using Hibernate as a persistence provider) will either use an existing sequence or create one when you generate the SQL schema automatically. If your DBMS doesn't support sequences, this will be the special “sequence table” name.

❺ You can start with an `initial_value` that gives you room for test data. For example, when your integration test runs, Hibernate (or Spring Data JPA using Hibernate as a persistence provider) will make any new data insertions from test code with identifier values greater than 1,000. Any test data you want to import before the test can use numbers 1 to 999, and you can refer to the stable identifier values in your tests: “Load item with id 123 and run some tests on it.” This is applied when Hibernate (or Spring Data JPA using Hibernate as a persistence provider) generates the SQL schema and sequence; it's a DDL option.

You can share the same database sequence among all your domain model classes. There is no harm in specifying `@GeneratedValue(generator = "ID_GENERATOR")` in all your entity classes. It doesn't matter if primary key values aren't contiguous for a particular entity, as long as they're unique within one table.

Finally, you can use `java.lang.Long` as the type of the identifier property in the entity class, which maps perfectly to a numeric database sequence generator. You could also use a `long` primitive. The main difference is what `someItem.getId()` returns on a new item that hasn't been stored in the database: either `null` or `0`. If you want to test whether an item is new, a `null` check is probably easier for someone else reading your code to understand. You shouldn't use another integral type, such as `int` or `short`, for identifiers. Although they will work for a while (perhaps even years), as your database size grows, you may be limited by their range. An `Integer` would work for almost two months if you generated a new identifier each millisecond with no gaps, and a `Long` would last for about 300 million years.

Although it is recommended for most applications, the `enhanced-sequence` strategy, as shown in listing 5.2, is just one of the strategies built into Hibernate. The key generator's configuration is unaware of the framework that uses it, and the programmer will never manage the value of the primary key. This is done at the level of the frameworks. The code will look like listings 5.3 and 5.4.

Listing 5.3 Persisting an `Item` with a generated primary key from Hibernate JPA

```
Path: Ch05/generator/src/test/java/com/manning/javapersistence/ch05
➡ /HelloWorldJPATest.java
```

```
em.getTransaction().begin();

Item item = new Item();
item.setName("Some Item");
item.setAuctionEnd(Helper.tomorrow());
em.persist(item);

em.getTransaction().commit();
```

Listing 5.4 Persisting an `Item` with a generated primary key from Spring Data JPA

```
Path: Ch05/generator/src/test/java/com/manning/javapersistence/ch05
➡ /HelloWorldSpringDataJPATest.java
```

```
Item item = new Item();
item.setName("Some Item");
item.setAuctionEnd(Helper.tomorrow());
itemRepository.save(item);
```

After running any of the Hibernate JPA or Spring Data JPA programs, a new `ITEM` will be inserted in the database with the `id` 1000, the first one specified by the generator (figure 5.5). The value to be generated for the next insertion is kept inside `JPWHSD_SEQUENCE` (figure 5.6).

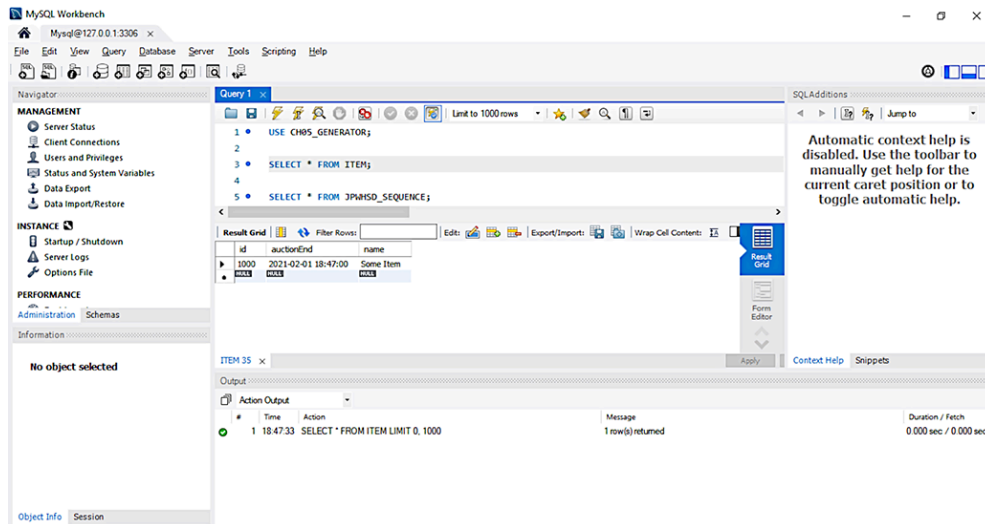


Figure 5.5 The content of the `ITEM` table after inserting a row with a generated primary key

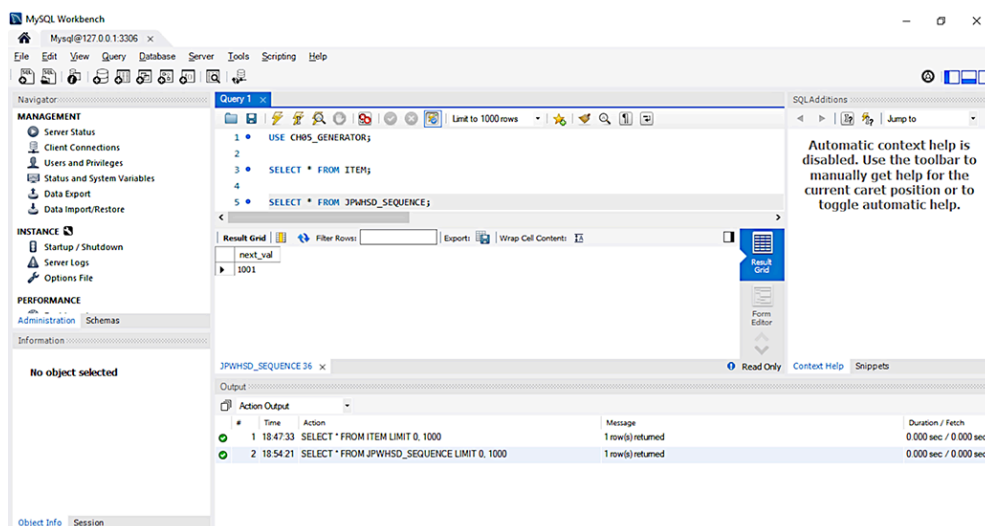


Figure 5.6 The next generated value kept by `JPWHSD_SEQUENCE`

5.2.5 Identifier generator strategies

Hibernate and Spring Data JPA using Hibernate as a provider offer several identifier generator strategies, and we'll list and discuss them in this section. We won't address the deprecated generator strategies.

If you don't want to read the whole list now, enable

`GenerationType.AUTO` and check what Hibernate defaults to for your database dialect. It's most likely `sequence` or `identity`—good choices, but maybe not the most efficient or portable choices. If you require consistent, portable behavior and identifier values that are available before `INSERT`s, use `enhanced-sequence`, as shown in the previous section.

This is a portable, flexible, and modern strategy that also offers various optimizers for large datasets.

Generating identifiers before or after `INSERT` : what's the difference?

An ORM service tries to optimize SQL `INSERT` s, such as by batching several at the JDBC level. Hence, SQL execution occurs as late as possible during a unit of work, and not when you call

`entityManager.persist(someItem)` . This merely queues the insertion for later execution and, if possible, assigns the identifier value.

However, if you now call `someItem.getId()` , you might get `null` back if the engine wasn't able to generate an identifier before the `INSERT` .

In general, we prefer *pre-insert* generation strategies that produce identifier values independently before `INSERT` . A common choice is to use a shared and concurrently accessible database sequence. Auto-incremented columns, column default values, and trigger-generated keys are available only after the `INSERT` .

Before we discuss the full list of identifier generator strategies, the recommendations for these strategies are as follows:

- In general, prefer pre-insert generation strategies that produce identifier values independently before `INSERT` .
- Use `enhanced-sequence` , which uses a native database sequence when that's supported and otherwise falls back to using an extra database table with a single column and row, emulating a sequence.

The following list outlines Hibernate's identifier generator strategies with their options and our usage recommendations. We also discuss the relationship between each standard JPA strategy and its native Hibernate equivalent. Hibernate has been growing organically, so there are now two sets of mappings between standard and native strategies; we refer to them as *old* and *new* in the list. You can switch this mapping with the `hibernate.id.new_generator_mappings` setting in your `persistence.xml` file. The default is `true` , which means the new mapping is used. Software doesn't age quite as well as wine.

- `native` —This option automatically selects a strategy, such as `sequence` or `identity` , depending on the configured SQL dialect. You have to look at the Javadoc (or even the source) of the SQL dialect you configured in `persistence.xml` to determine which strategy will be selected. This is equivalent to JPA `GenerationType.AUTO` with the old mapping.

- **sequence** —This strategy uses a native database sequence named `HIBERNATE_ SEQUENCE`. The sequence is called before each `INSERT` of a new row. You can customize the sequence name and provide additional DDL settings; see the Javadoc for the `org.hibernate.id.SequenceGenerator` class.
- **enhanced-sequence** —This strategy uses a native database sequence when it's supported; otherwise it falls back to using an extra database table with a single column and row, emulating a sequence (the default table name is `HIBERNATE_ SEQUENCE`). Using this strategy always calls the database “sequence” before an `INSERT`, providing the same behavior independently of whether the DBMS supports real sequences. This strategy also supports an `org.hibernate.id.enhanced.Optimizer` to avoid hitting the database before each `INSERT`, and it defaults to no optimization and fetching a new value for each `INSERT`. This is equivalent to JPA `GenerationType.SEQUENCE` and `GenerationType.AUTO` with the new mapping enabled, and it's probably your best option of the built-in strategies. For all the parameters, see the Javadoc for the `org.hibernate.id.enhanced.SequenceStyleGenerator` class.
- **enhanced-table** —This strategy uses an extra table named `HIBERNATE_ SEQUENCES`, with one row by default representing the sequence and storing the next value. This value is selected and updated when an identifier value has to be generated. You can configure this generator to use multiple rows instead: one for each generator (see the Javadoc for `org.hibernate.id.enhanced.TableGenerator`). This is equivalent to JPA `GenerationType.TABLE` with the new mapping enabled. It replaces the outdated but similar `org.hibernate.id.MultipleHiLoPerTableGenerator`, which was the old mapping for JPA `GenerationType.TABLE`.
- **identity** —This strategy supports `IDENTITY` and auto-increment columns in DB2, MySQL, MS SQL Server, and Sybase. The identifier value for the primary key column will be generated on the `INSERT` of a row. It has no options. Unfortunately, due to a quirk in Hibernate's code, you *cannot* configure this strategy in `@GenericGenerator`. The DDL generation will not include the identity or auto-increment option for the primary key column. The only way to use it is with JPA `GenerationType.IDENTITY` and the old or new mapping, making it the default for `GenerationType.IDENTITY`.
- **increment** —At Hibernate startup, this strategy reads the maximum (numeric) primary key column value of each entity's table and increments the value by one each time a new row is inserted. This is especially efficient if a non-clustered Hibernate application has exclusive access to the database, but don't use it in any other scenario.

- `select` —With this strategy, Hibernate won't generate a key value or include the primary key column in an `INSERT` statement. Hibernate expects the DBMS to assign a value to the column on insertion (the default value in the schema or the value given by a trigger). Hibernate then retrieves the primary key column with a `SELECT` query after insertion. The required parameter is `key`, naming the database identifier property (such as `id`) for the `SELECT`. This strategy isn't very efficient and should only be used with old JDBC drivers that can't return generated keys directly.
- `uuid2` —This strategy produces a unique 128-bit UUID in the application layer. This is useful when you need globally unique identifiers across databases (such as if you merge data from several distinct production databases in batch runs every night into an archive). The UUID can be encoded either as a `java.lang.String`, a `byte[16]`, or a `java.util.UUID` property in your entity class. This replaces the legacy `uuid` and `uuid.hex` strategies. You configure it with an `org.hibernate.id.UUIDGenerationStrategy`; see the Javadoc for the `org.hibernate.id.UUIDGenerator` class for more details.
- `guid` —This strategy uses a globally unique identifier produced by the database, with an SQL function available on Oracle, Ingres, MS SQL Server, and MySQL. Hibernate calls the database function before an `INSERT`. The value maps to a `java.lang.String` identifier property. If you need full control over identifier generation, configure the strategy of `@GenericGenerator` with the fully qualified name of a class that implements the `org.hibernate.id.IdentityGenerator` interface.

If you haven't already, go ahead and add identifier properties to the entity classes of your domain model. Make sure you do not expose the identifier outside the business logic, such as through an API—this identifier has no business logic meaning and it is related only to the persistence.

After you complete the basic mapping of each entity and its identifier property, you can continue to map the value-typed properties of the entities. We'll talk about value-type mappings in the next chapter. First, though, read on for some special options that can simplify and enhance your class mappings.

5.3 Entity-mapping options

You've now mapped a persistent class with `@Entity`, using defaults for all the other settings, such as the mapped SQL table name. We'll now explore some class-level options and how you can control them:

- Naming defaults and strategies

- Dynamic SQL generation
- Entity mutability

These are options. If you prefer, you can skip this section for now, and come back later when you have to deal with specific problems.

5.3.1 Controlling names

Let's talk about the naming of entity classes and tables. If you only specify `@Entity` on a persistence-capable class, the default mapped table name will be the same as the class name. For example, the Java entity class `Item` maps to the `ITEM` table. A Java entity class named `BidItem` will map to the `BID_ITEM` table (here, the camel case will be converted into snake case). You can override the table name with the JPA `@Table` annotation, as shown next. (See the `mapping` folder for the source code.)

NOTE We write SQL artifact names in `UPPERCASE` to make them easier to distinguish—SQL is actually case insensitive.

Listing 5.5 Overriding the mapped table name with the `@Table` annotation

```
Path: Ch05/mapping/src/main/java/com/manning/javapersistence/ch05/model
➡ /User.java

@Entity

@Table(name = "USERS")

public class User {
    // . . .
}
```

The `User` entity would map to the `USER` table, but this is a reserved keyword in most SQL DBMSs, so you can't have a table with that name. Instead we have mapped it to `USERS`. The `@javax.persistence.Table` annotation also has `catalog` and `schema` options if your database layout requires these as naming prefixes.

If you really have to, quoting allows you to use reserved SQL names and even work with case-sensitive names.

QUOTING SQL IDENTIFIERS

From time to time, especially in legacy databases, you'll encounter identifiers with strange characters or whitespace, or you'll wish to force case sensitivity. Or, as in the previous example, the automatic mapping of a

class or property would require a table or column name that is a reserved keyword. Hibernate and Spring Data JPA using Hibernate as a provider know the reserved keywords of your DBMS through the configured database dialect, and they can automatically put quotes around such strings when generating SQL. You can enable this automatic quoting with `hibernate .auto_quote_ keyword=true` in your persistence unit configuration. If you're using an older version of Hibernate, or you find that the dialect's information is incomplete, you must still apply quotes on names manually in your mappings if there is a conflict with a keyword.

If you quote a table or column name in your mapping with backticks, Hibernate always quotes this identifier in the generated SQL. This still works in the latest versions of Hibernate, but JPA 2.0 standardized this functionality as *delimited identifiers* with double quotes.

This is the Hibernate-only quoting with backticks, modifying the previous example:

```
@Table(name = "`USER`")
```

To be JPA-compliant, you also have to escape the quotes in the string:

```
@Table(name = "\"USER\"")
```

Either way works fine with Hibernate and Spring Data JPA using Hibernate as a provider. It knows the native quote character for your dialect and now generates SQL accordingly: `[USER]` for MS SQL Server, `'USER'` for MySQL, `"USER"` for H2, and so on.

If you have to quote *all* SQL identifiers, create an `orm.xml` file and add the setting `<delimited-identifiers/>` to its `<persistence-unit-defaults>` section, as you saw earlier in listing 3.8. Hibernate then enforces quoted identifiers everywhere.

You should consider renaming tables or columns with reserved keyword names whenever possible. Ad hoc SQL queries are difficult to write in an SQL console if you have to quote and escape everything properly by hand. Also, you should avoid using quoted identifiers for databases that are also accessed by other means than Hibernate, JPA, or Spring Data (such as for reporting). Having to use delimiters for all identifiers in a (complex) report query is really painful.

Next, let's see how Hibernate and Spring Data JPA with Hibernate as a provider can help when you encounter organizations with strict conven-

tions for database table and column names.

IMPLEMENTING NAMING CONVENTIONS

Hibernate provides a feature that allows you to enforce naming standards automatically. Suppose that all table names in CaveatEmptor should follow the pattern `CE_<table name>`. One solution is to manually specify the `@Table` annotation on all entity classes, but this approach is time-consuming and easily forgotten. Instead, you can implement Hibernate's `PhysicalNamingStrategy` interface or override an existing implementation, as in the following listing.

Listing 5.6 Overriding default naming conventions with `PhysicalNamingStrategy`

```
Path: Ch05/mapping/src/main/java/com/manning/javapersistence/ch05
➡ /CENamingStrategy.java

public class CENamingStrategy extends PhysicalNamingStrategyStandardIn

    @Override

    public Identifier toPhysicalTableName(Identifier name,

                                           JdbcEnvironment context) {

        return new Identifier("CE_" + name.getText(), name.isQuoted())

    }

}
```

The overridden `toPhysicalTableName()` method prepends `CE_` to all generated table names in your schema. Look at the Javadoc of the `PhysicalNamingStrategy` interface; it offers methods for custom naming of columns, sequences, and other artifacts.

You have to enable the naming strategy implementation. With Hibernate JPA, this is done in `persistence.xml`:

```
Path: Ch05/mapping/src/main/resources/META-INF/persistence.xml

<persistence-unit name="ch05.mapping">
    ...
    <properties>
        ...
        <property name="hibernate.physical_naming_strategy"
            value="com.manning.javapersistence.ch05.CENamingStrate
```

```
</properties>
</persistence-unit>
```

With Spring Data JPA using Hibernate as a persistence provider, this is done from the `LocalContainerEntityManagerFactoryBean` configuration:

```
Path: Ch05/mapping/src/test/java/com/manning/javapersistence/ch05
➡ /configuration/SpringDataConfiguration.java

properties.put("hibernate.physical_naming_strategy",
               CENamingStrategy.class.getName());
```

Let's now take a quick look at another related problem, the naming of entities for queries.

NAMING ENTITIES FOR QUERYING

By default, all entity names are automatically imported into the namespace of the query engine. In other words, you can use short class names without a package prefix in JPA query strings, which is convenient:

```
Path: Ch05/generator/src/test/java/com/manning/javapersistence/ch05
➡ /HelloWorldJPATest.java

List<Item> items = em.createQuery("select i from Item i",
➡ Item.class).getResultList();
```

This only works when you have one `Item` class in your persistence unit. If you add another `Item` class in a different package, you should rename one of them for JPA if you want to continue using the short form in queries:

```
package my.other.model;
@javax.persistence.Entity(name = "AuctionItem")
public class Item {
    // . . .
}
```

The short query form is now `select i from AuctionItem i` for the `Item` class in the `my.other.model` package. Thus you resolve the naming conflict with another `Item` class in another package. Of course, you can always use fully qualified long names with the package prefix.

This completes our tour of the naming options. Next we'll discuss how Hibernate and Spring Data JPA using Hibernate generate the SQL that contains these names.

5.3.2 Dynamic SQL generation

By default, Hibernate and Spring Data JPA using Hibernate as a provider create SQL statements for each persistent class when the persistence unit is created on startup. These statements are simple create, read, update, and delete (CRUD) operations for reading a single row, deleting a row, and so on. It's cheaper to create and cache them instead of generating SQL strings every time such a simple query has to be executed at runtime. Besides, prepared statement caching at the JDBC level is much more efficient if there are fewer statements.

How can Hibernate create an `UPDATE` statement on startup? After all, the columns to be updated aren't known at this time. The answer is that the generated SQL statement updates all columns, and if the value of a particular column isn't modified, the statement sets it to its old value.

In some situations, such as a legacy table with hundreds of columns, where the SQL statements will be large for even the simplest operations (such as when only one column needs updating), you should disable this startup SQL generation and switch to dynamic statements generated at runtime. An extremely large number of entities can also influence startup time because Hibernate has to generate all the SQL statements for CRUD up front. Memory consumption for this query statement cache will also be high if a dozen statements must be cached for thousands of entities. This can be a concern in virtual environments with memory limitations or on low-power devices.

To disable the generation of `INSERT` and `UPDATE` SQL statements on startup, you need to use native Hibernate annotations:

```
@Entity
@org.hibernate.annotations.DynamicInsert
@org.hibernate.annotations.DynamicUpdate
public class Item {
    // . . .
}
```

By enabling dynamic insertion and updates, you tell Hibernate to produce the SQL strings when needed, not up front. The `UPDATE` will only contain columns with updated values, and the `INSERT` will only contain non-nullable columns.

5.3.3 Making an entity immutable

Instances of a particular class may be immutable. For example, in `CaveatEmptor`, a `Bid` made for an item is immutable. Hence, Hibernate or Spring Data JPA using Hibernate as a provider never needs to execute `UPDATE` statements on the `BID` table. Hibernate can also make a few other optimizations, such as avoiding dirty checking if you map an immutable class, as shown in the next example. Here the `Bid` class is immutable, and instances are never modified:

```
@Entity
@org.hibernate.annotations.Immutable
public class Bid {
    // . . .
}
```

A POJO is immutable if no public setter methods for any properties of the class are exposed—all values are set in the constructor. Hibernate or Spring Data JPA using Hibernate as a provider should access the fields directly when loading and storing instances. We talked about this earlier in this chapter: if the `@Id` annotation is on a field, Hibernate will access the fields directly, and you are free to design your getter and setter methods. Also, remember that not all frameworks work with POJOs without setter methods.

When you can't create a view in your database schema, you can map an immutable entity class to an SQL `SELECT` query.

5.3.4 Mapping an entity to a subselect

Sometimes your DBA won't allow you to change the database schema. Even adding a new view might not be possible. Let's say you want to create a view that contains the identifier of an auction `Item` and the number of bids made for that item (see the `subselect` folder for the source code). Using a Hibernate annotation, you can create an application-level view, a read-only entity class mapped to an SQL `SELECT`:

```
Path: Ch05/subselect/src/main/java/com/manning/javapersistence/ch05/mc
➡ /ItemBidSummary.java
```

```
@Entity
@org.hibernate.annotations.Immutable
@org.hibernate.annotations.Subselect(
    value = "select i.ID as ITEMID, i.NAME as NAME, " +
            "count(b.ID) as NUMBEROFBIDS " +
            "from ITEM i left outer join BID b on i.ID = b.ITEM_ID " +
```

```

        "group by i.ID, i.NAME"
    )
    @org.hibernate.annotations.Synchronize({"ITEM", "BID"})
    public class ItemBidSummary {
        @Id
        private Long itemId;
        private String name;
        private long numberOfBids;
        public ItemBidSummary() {
        }
        // Getter methods . . .
        // . . .
    }

```

You should list all the table names referenced in your `SELECT` in the `@org.hibernate.annotations.Synchronize` annotation. The framework will then know it has to flush modifications of `Item` and `Bid` instances before it executes a query against `ItemBidSummary`. If there are in-memory modifications that haven't yet been persisted to the database but that may affect the query, Hibernate (or Spring Data JPA using Hibernate as a provider) detects this and flushes the changes before executing the query. Otherwise the result may be a stale state. As there is no `@Table` annotation on the `ItemBidSummary` class, the framework doesn't know when it must auto-flush before executing a query. The `@org.hibernate.annotations.Synchronize` annotation indicates that the framework needs to flush the `ITEM` and `BID` tables before executing the query.

Using the read-only `ItemBidSummary` entity class from Hibernate JPA will look like this:

```

Path: Ch05/subselect/src/test/java/com/manning/javapersistence/ch05
➡ /ItemBidSummaryTest.java

TypedQuery<ItemBidSummary> query =
    em.createQuery("select ibs from ItemBidSummary ibs where
➡ ibs.itemId = :id",
        ItemBidSummary.class);
ItemBidSummary itemBidSummary =
    query.setParameter("id", 1000L).getSingleResult();

```

To use the read-only `ItemBidSummary` entity class from Spring Data JPA, you'll first need to introduce a new Spring Data repository:

```

Path: Ch05/mapping/src/main/java/com/manning/javapersistence/ch05
➡ /repositories/ItemBidSummaryRepository.java

```

```
public interface ItemBidSummaryRepository extends
    CrudRepository<ItemBidSummary, Long> {
}
```

The repository will be effectively used like this:

```
Path: Ch05/subselect/src/test/java/com/manning/javapersistence/ch05
➡ /ItemBidSummarySpringDataTest.java
```

```
Optional<ItemBidSummary> itemBidSummary =
    itemBidSummaryRepository.findById(1000L);
```

Summary

- Entities are coarser-grained classes of a system. Their instances have independent lifecycles and their own identities, and many other instances can reference them.
- Value types are dependent on a particular entity class. A value type instance is bound to its owning entity instance, and only one entity instance can reference it—it has no individual identity.
- Java identity, object equality, and database identity are different concepts: the first two apply in the object-oriented world and the last one in the relational database world.
- A good primary key is never null, is unique, and never changes.
- Generators for primary keys can be configured using different strategies.
- You can use entities, mapping options, and naming strategies both from Hibernate JPA and from Spring Data JPA.